Copyright All rights reserved

19. Modern C99~23

(C99~C23)

Heesung Oh

https://github.com/3dapi

19.1 C99 — 현대 C의 출발점

Heesung Oh
Copyright All rights reserved

- Modern C
  - C99 → C11 → C23 발전

- 핵심 변화
  - 데이터 표현 정확성
  - 멀티스레드 안정성
  - 정렬 및 메모리 제어
  - 타입 안전성

19.1 C99 – 개요

Heesung Oh
Copyright All rights reserved

- C99
  - 현대 C의 출발점

- 주요 기능
  - 정확한 정수 타입
  - bool 타입
  - 구조체 확장
  - 메모리 표현 강화

19.1 C99

Heesung Oh
Copyright All rights reserved

- ⟨stdint.h⟩
  - 고정 폭 정수 타입: int8_t, int16_t, int32_t, int64_t
  - 정확한 비트 크기 → 플랫폼 독립
  - 용도: 파일 포맷, 네트워크 패킷, GPU 버퍼

- ⟨stdbool.h⟩
  - bool 타입: bool, true, false
  - 이전 방식: int / 매크로 사용
  - 가독성 향상, 의도 명확화

19.1 C99 – Flexible Array Member

Heesung Oh
Copyright All rights reserved

- Flexible Array Member
  - 가변 길이 구조체 멤버
  - 런타임 크기 결정
  - 연속 메모리 구조
  - 용도: 패킷, 바이너리 데이터

```
typedef struct Packet
{
    uint32_t size;
    uint8_t  data[];
} Packet;
```

- sizeof(Packet)
  - data 제외
  - 구조체 기본 크기만 포함
  - 패딩 포함 가능
  - data 시작 위치 기준 → 추가 메모리 할당 필요

```
// 100바이트의 데이터를 담고 싶은 경우
uint32_t dataSize = 100;
Packet* p = (Packet*)malloc(sizeof(Packet) + dataSize);
p->size = dataSize; // p->data[0]~data[99] 접근 가능
```

19.1 C99 – VLA, 지정 초기화

Heesung Oh
Copyright All rights reserved

- 가변 길이 배열
  - int buffer[n];
  - 런타임 크기 결정
  - 스택 메모리 사용
  - C11 이후 선택(Optional) 기능
    → 크로스 플랫폼 개발에서는 사용을 지양

- 지정 초기화
  - 필드 이름 기반 초기화
  - 구조 변경에 안전
  - Ex)

```
struct Vec2 p = { .x = 10.0f, .y = 20.0f };
```

19.1 C99 – 복합 리터럴

Heesung Oh
Copyright All rights reserved

- 복합 리터럴
  - 임시 객체 안전 사용
  - 즉시 사용 가능
  - 유효 범위: 블록(scope) 내, 블록 종료 시 소멸
  - 용도: 함수 인자 전달

```
int* p = (int[]){1, 2, 3};
```

```
printVector((struct Vec2){1.0f, 2.0f});
```

19.1 C99 – restrict

Heesung Oh
Copyright All rights reserved

- 포인터 별칭 금지
- 컴파일러 최적화 가능
- 자동 벡터화 → 수치 계산 성능 향상

```
void add(float* restrict a, float* restrict b, float* restrict c, int n)
{
    for(int i = 0; i < n; ++i)
        c[i] = a[i] + b[i];
}
```

19.1 C99 – inline

Heesung Oh
Copyright All rights reserved

- 함수 인라인 확장 힌트
- 함수 호출 제거 가능
- 성능 최적화 → 컴파일러가 판단

```
inline int square(int x)
{
    return x * x;
}
```

19.2 C11 — 멀티스레드와 안전성

Heesung Oh
Copyright All rights reserved

- C11
  - 멀티스레드와 안전성 강화
  - 컴파일 타임 검증
  - 원자 연산
  - 정렬 제어

19.2 C11 – 멀티스레드와 안전성

Heesung Oh
Copyright All rights reserved

- _Static_assert
  - 컴파일 타임 검증
  - 용도: 구조 크기 검증, 데이터 정합성 확인

```
_Static_assert(sizeof(float) == 4, "float size error");
```

- 익명 구조체 / 공용체
  - 동일 데이터 다양한 접근
  - 용도: 수학 라이브러리

```
typedef struct Vec3
{
    union {
        struct { float x, y, z; };
        float v[3];
    };
} Vec3;
```

19.2 C11 – _Generic / _Atomic

Heesung Oh
Copyright All rights reserved

- _Generic
  - 타입 기반 분기
  - 컴파일 타임 선택
  - 함수 오버로딩 유사
    → 매크로 확장

```
#define my_sin(x) _Generic((x), \
  float: sinf, \
  double: sin, \
  long double: sinl)(x)
```

- _Atomic
  - 원자적 연산
  - 데이터 경쟁 방지
  - 멀티스레드 안전성
    → Lock-free 구조

```
_Atomic int counter;
```

19.2 C11 – 정렬 / Annex K

Heesung Oh
Copyright All rights reserved

- 정렬과 메모리
  - _Alignas
  - aligned_alloc
  - SIMD 대응
  - 메모리 정렬 보장
    → 성능 중요

```
_Alignas(16) float
matrix[4][4];
```

```
void* p =
aligned_alloc(16, 1024);
```

- Annex K
  - _s 가 붙은 함수
  - 보안 함수, 안전한 API, 선택적 기능
  - strcpy_s, strcat_s, sprintf_s, memcpy_s 추가

19.3 C23 — 최신 Modern C

Heesung Oh
Copyright All rights reserved

- C23
  - 최신 Modern C
  - 표현력 증가
  - 타입 안전성 강화
  - 문법 개선

19.3 C23 — 최신 Modern C

Heesung Oh
Copyright All rights reserved

- nullptr
  - NULL 대체, 타입 안전성 향상→ 포인터 표현 개선
- 키워드 승격 (언어 통합)
  - bool, static_assert, alignas
- #embed
  - 바이너리 포함, 컴파일 타임 삽입
  - 용도: 셰이더, 리소스 데이터

```
const uint8_t shader_code[] = {
    #embed "triangle.spv"
};
```

- Empty Initializer
  - 모든 값 0 초기화 → 간결한 코드

```
struct Vec2 v = {};
```

- typeof
  - 타입 추론
  - 컴파일러 확장, 매크로 간결화

19.3 C23 — 최신 Modern C

Heesung Oh
Copyright All rights reserved

- 숫자 표현 개선
  - 가독성 향상

```
int money = 1'000'000;
int mask  = 0b11001100;
```

- _BitInt
  - 정확한 비트 정수
  - 비트 단위 제어
  - 메모리 최적화 → 프로토콜/압축

```
_BitInt(5) a = 17;
```

- 확인된 정수 연산
  - ckd_add
  - 오버플로우 감지 → 안전한 계산

```
#include <stdckdint.h>
ckd_add(&result, a, b);
```

학습 정리

Heesung Oh
Copyright All rights reserved

- Modern C 핵심
  - 정확한 데이터 표현
  - 멀티스레드 안정성
  - 메모리 제어
  - → 고성능 시스템 개발 기반